

```
import matplotlib.pyplot as plt
plt.rcParams['figure.figsize'] = (6,4)
plt.rcParams['figure.dpi'] = 100
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error

url = "https://raw.githubusercontent.com/aravind9722/datasets-for-ML-projects/main/cardekho_dataset.csv"
df = pd.read_csv(url)
print(df.shape)
df.head()
```

(15411, 14)

	Unnamed: 0	car_name	brand	model	vehicle_age	km_driven	seller_type	fuel_type	transmission_type	mileage	engine
0	0	Maruti Alto	Maruti	Alto	9	120000	Individual	Petrol	Manual	19.70	796
1	1	Hyundai Grand	Hyundai	Grand	5	20000	Individual	Petrol	Manual	18.90	1197
2	2	Hyundai i20	Hyundai	i20	11	60000	Individual	Petrol	Manual	17.00	1197

Next steps: [Generate code with df](#) [New interactive sheet](#)

##Q1. What is the average selling price of cars in this dataset?

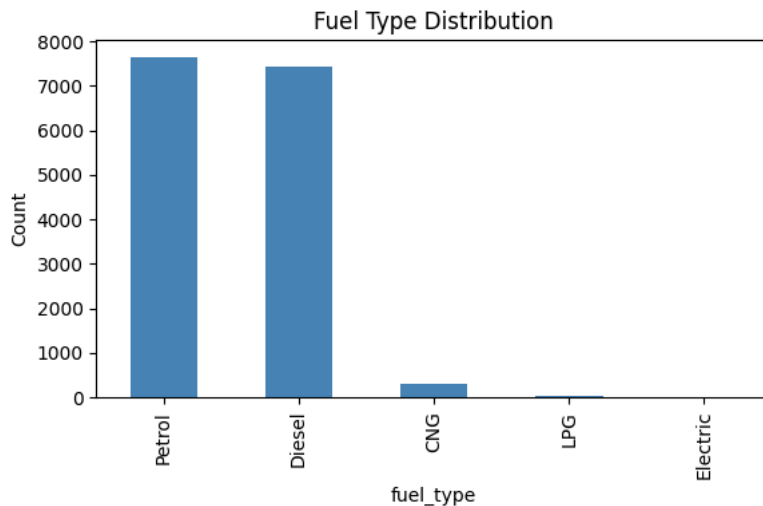
```
avg_price = df['selling_price'].mean()
print(f"Average Selling Price: ₹{avg_price:,.2f}")
```

Average Selling Price: ₹774,971.12

##Q2. Which fuel type is most common?

```
fuel_counts = df['fuel_type'].value_counts()
print(fuel_counts)
fuel_counts.plot(kind='bar', color='steelblue', title='Fuel Type Distribution')
plt.ylabel('Count')
plt.tight_layout()
plt.show()
```

```
fuel_type
Petrol    7643
Diesel    7419
CNG       301
LPG        44
Electric     4
Name: count, dtype: int64
```



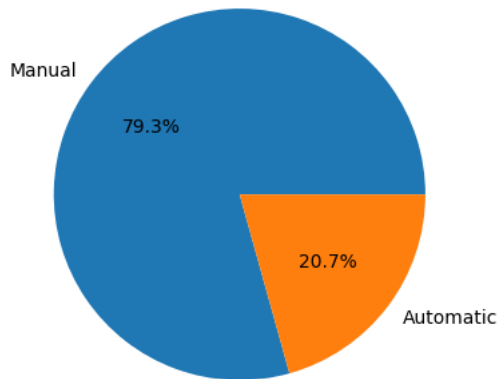
##Q3. How many cars have manual vs automatic transmission?

```
transmission_counts = df['transmission_type'].value_counts()
```

```
print(transmission_counts)
transmission_counts.plot(kind='pie', autopct='%1.1f%%', title='Transmission Type')
plt.ylabel('')
plt.tight_layout()
plt.show()
```

```
transmission_type
Manual      12225
Automatic   3186
Name: count, dtype: int64
```

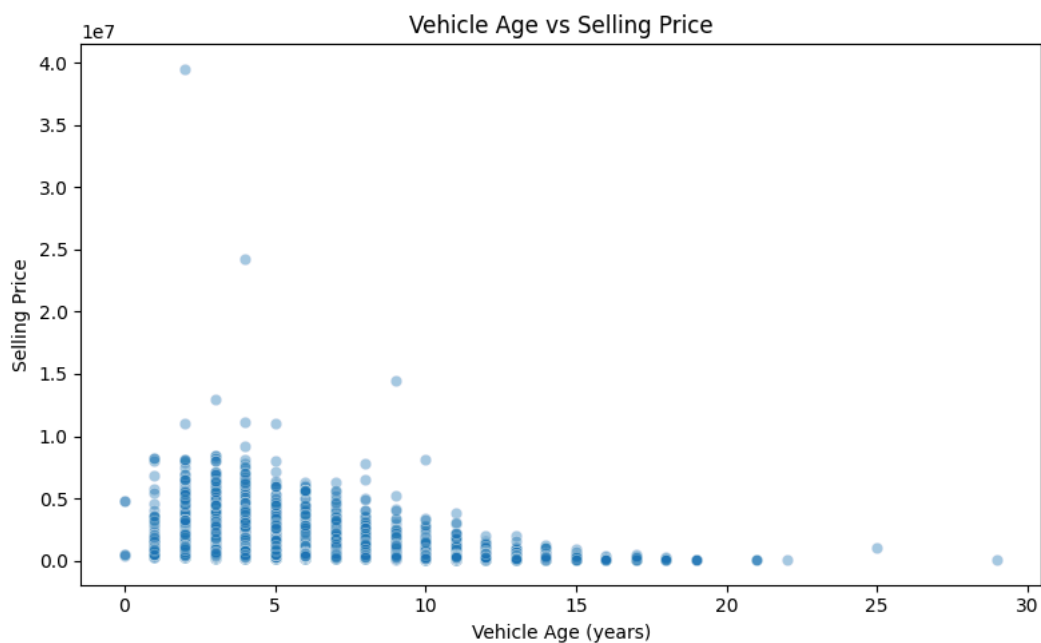
Transmission Type



##Q4. Does vehicle age affect selling price?

```
plt.figure(figsize=(8,5))
sns.scatterplot(data=df, x='vehicle_age', y='selling_price', alpha=0.4)
plt.title('Vehicle Age vs Selling Price')
plt.xlabel('Vehicle Age (years)')
plt.ylabel('Selling Price')
plt.tight_layout()
plt.show()
```

```
print("Correlation:", df['vehicle_age'].corr(df['selling_price']))
```



Correlation: -0.2418514586696868

##Q5. Which brand has the highest average selling price?

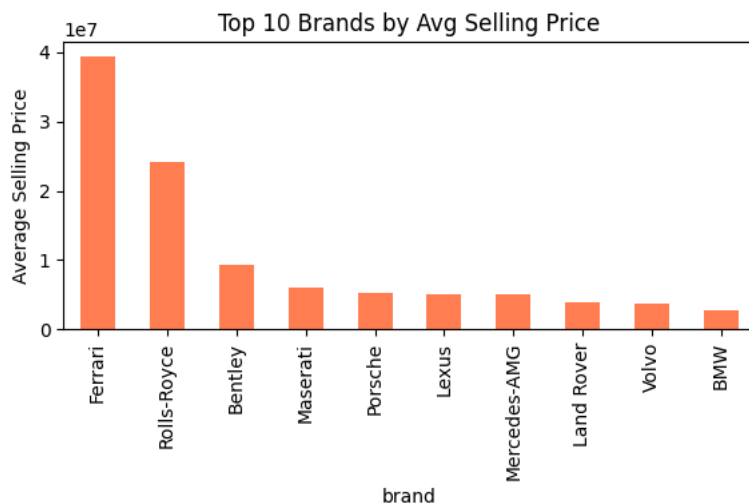
```
brand_avg = df.groupby('brand')['selling_price'].mean().sort_values(ascending=False).head(10)
print(brand_avg)
```

```
brand_avg.plot(kind='bar', color='coral', title='Top 10 Brands by Avg Selling Price')
plt.ylabel('Average Selling Price')
plt.tight_layout()
plt.show()
```

```

brand
Ferrari      3.950000e+07
Rolls-Royce  2.420000e+07
Bentley      9.266667e+06
Maserati     6.100000e+06
Porsche     5.161190e+06
Lexus       5.146500e+06
Mercedes-AMG 5.100000e+06
Land Rover  3.823902e+06
Volvo       3.729700e+06
BMW         2.693827e+06
Name: selling_price, dtype: float64

```



##Q6. Can we predict selling price based on mileage, engine size, and max power?

```

features = ['mileage', 'engine', 'max_power']
model_df = df[features + ['selling_price']].dropna()

X = model_df[features]
y = model_df['selling_price']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

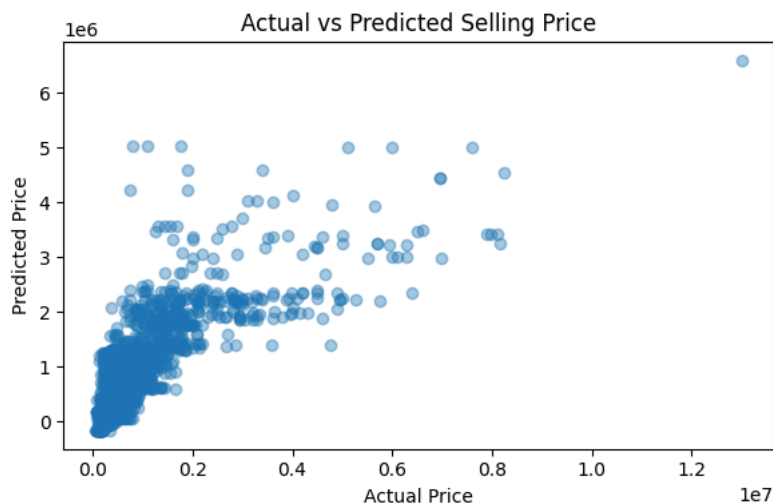
y_pred = model.predict(X_test)

print(f"R² Score: {r2_score(y_test, y_pred):.4f}")
print(f"MAE: ₹{mean_absolute_error(y_test, y_pred):.2f}")

plt.scatter(y_test, y_pred, alpha=0.4)
plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')
plt.title('Actual vs Predicted Selling Price')
plt.tight_layout()
plt.show()

```

R² Score: 0.6062
MAE: ₹301,025.52



##Q7. Which seller type offers the cheapest cars on average?

```
seller_avg = df.groupby('seller_type')['selling_price'].mean().sort_values()
print(seller_avg)

seller_avg.plot(kind='bar', color='teal', title='Avg Selling Price by Seller Type')
plt.ylabel('Average Selling Price')
plt.tight_layout()
plt.show()
```

```
seller_type
Trustmark Dealer    571959.537572
Individual          617880.483418
Dealer              872505.503722
Name: selling_price, dtype: float64
```

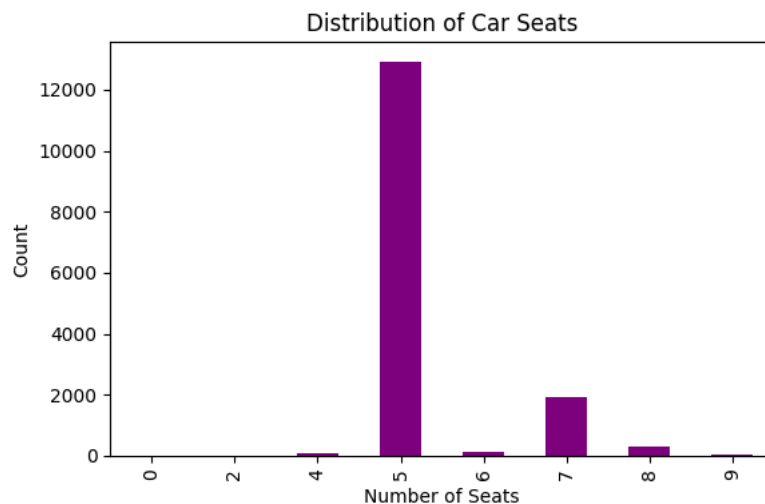


##Q8. What is the distribution of car seats?

```
seat_dist = df['seats'].value_counts().sort_index()
print(seat_dist)

seat_dist.plot(kind='bar', color='purple', title='Distribution of Car Seats')
plt.xlabel('Number of Seats')
plt.ylabel('Count')
plt.tight_layout()
plt.show()
```

```
seats
0      2
2      7
4     77
5    12910
6     127
7    1922
8     311
9      55
Name: count, dtype: int64
```



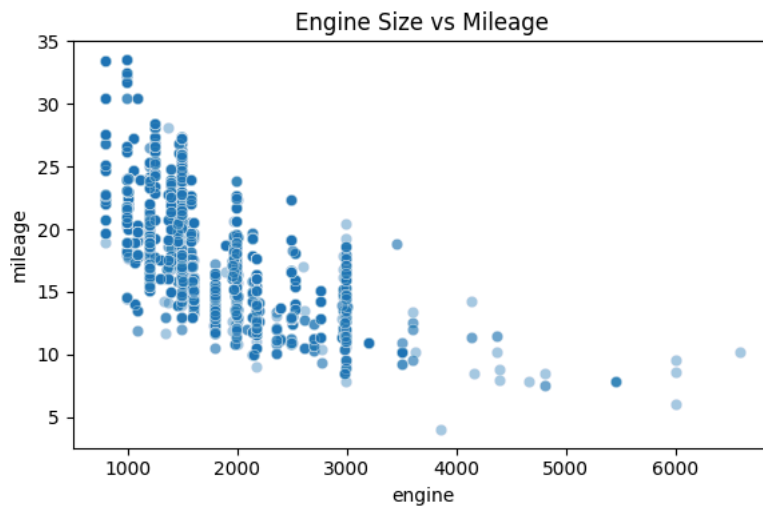
##Q9. Is there a correlation between engine size and mileage?

```
corr_df = df[['engine', 'mileage']].dropna()

print("Correlation:", corr_df['engine'].corr(corr_df['mileage']))

sns.scatterplot(data=corr_df, x='engine', y='mileage', alpha=0.4)
plt.title('Engine Size vs Mileage')
plt.tight_layout()
plt.show()
```

Correlation: -0.632987074009182

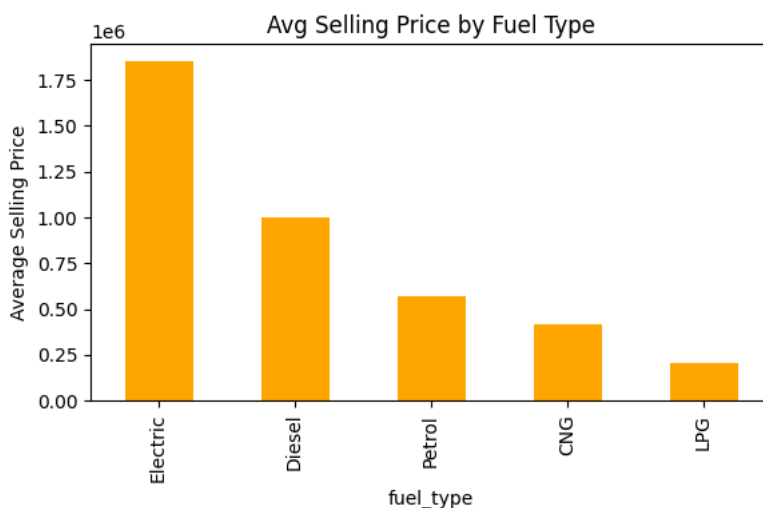


##Q10. What's the average selling price by fuel type?

```
fuel_price = df.groupby('fuel_type')['selling_price'].mean().sort_values(ascending=False)
print(fuel_price)

fuel_price.plot(kind='bar', color='orange', title='Avg Selling Price by Fuel Type')
plt.ylabel('Average Selling Price')
plt.tight_layout()
plt.show()
```

```
fuel_type
Electric    1.853500e+06
Diesel      1.000469e+06
Petrol      5.728619e+05
CNG         4.176877e+05
LPG         2.062727e+05
Name: selling_price, dtype: float64
```



##Q11. How does transmission type affect mileage?

```
trans_mileage = df[['transmission_type', 'mileage']].dropna()

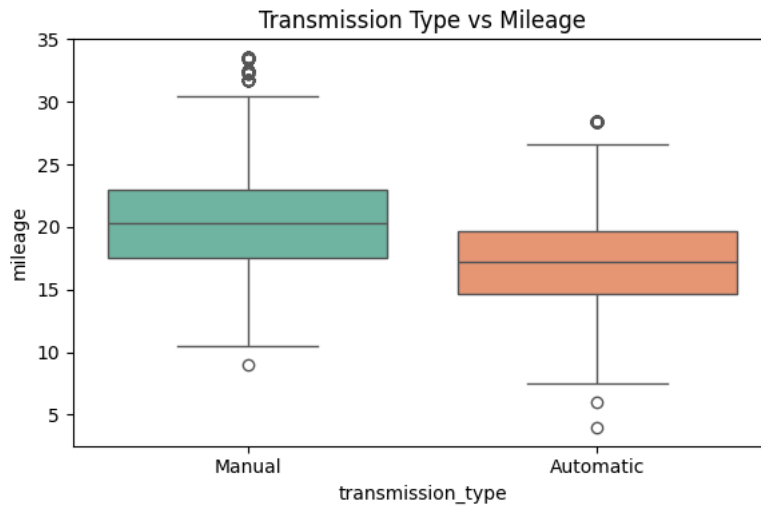
sns.boxplot(data=trans_mileage, x='transmission_type', y='mileage', palette='Set2')
plt.title('Transmission Type vs Mileage')
plt.tight_layout()
plt.show()

print(trans_mileage.groupby('transmission_type')['mileage'].mean())
```

```
/tmp/ipykernel_24087/2596437611.py:5: FutureWarning:
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and
```

```
sns.boxplot(data=trans_mileage, x='transmission_type', y='mileage', palette='Set2')
```



```
transmission_type
Automatic    17.391008
Manual       20.303206
Name: mileage, dtype: float64
```